SMART TRANSPORT SIMULATION

TEAM

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct {
    int vehicles;
    int capacity;
    float congestion;
} Road;

typedef struct {
    int green_time;
    char direction[10];
} TrafficLight;

Road roads[4];
TrafficLight lights[2];
int total_spawned = 0;
int total_completed = 0;

void adaptive_traffic_lights() {
    printf("\n=== FEATURE 1: ADAPTIVE TRAFFIC LIGHTS ===\n");

    int intersection, queue_length;

    printf("Enter intersection number (1 or 2): ");
    scanf("%d", &intersection);

    printf("Enter queue length (number of vehicles waiting): ");
    scanf("%d", &queue_length);

    int green_time = 30 + (queue_length / 10) * 5;
    if(green_time > 60) green_time = 60;

    printf("\n--- RESULTS ---\n");
    printf("Intersection: %d\n", intersection);
    printf("Queue Length: %d vehicles\n", queue_length);
    printf("Base Green Time: 30 seconds\n");
    printf("Extension: %d seconds\n", (queue_length / 10) * 5);
    printf("Final Green Time: %d seconds\n", green_time);

    if(queue_length < 10) {
        printf("Status: Low traffic - Standard timing\n");
    } else if(queue_length < 30) {
        printf("Status: Moderate traffic - Extended timing\n");
    } else {
        printf("Status: Heavy traffic - Maximum timing\n");
    }
}

void congestion_calculation() {
    printf("\n=== FEATURE 2: CONGESTION CALCULATION ===\n");

    int road_num, vehicles, capacity;

    printf("Enter road number (1-4): ");
    scanf("%d", &road_num);

    printf("Enter current vehicles on road: ");
    scanf("%d", &vehicles);

    printf("Enter road capacity: ");
    scanf("%d", &capacity);

    float congestion = ((float)vehicles / capacity) * 100.0;

    printf("\n--- RESULTS ---\n");
    printf("Road: %d\n", road_num);
    printf("Current Vehicles: %d\n", vehicles);
    printf("Road Capacity: %d\n", capacity);
    printf("Congestion Score: %.2f%%\n", congestion);

    if(congestion < 30) {
        printf("Congestion Level: LOW [GREEN]\n");
        printf("Traffic Flow: Smooth\n");
    } else if(congestion < 60) {
        printf("Congestion Level: MEDIUM [YELLOW]\n");
        printf("Traffic Flow: Moderate\n");
    } else {
        printf("Congestion Level: HIGH [RED]\n");
        printf("Traffic Flow: Heavy congestion\n");
    }
}

void rush_hour_simulation() {
    printf("\n=== FEATURE 3: RUSH HOUR SIMULATION ===\n");

    int hour;

    printf("Enter current hour (0-23): ");
    scanf("%d", &hour);

    float multiplier;
    int spawn_count;
    char phase[30];

    if(hour >= 7 && hour < 9) {
        multiplier = 2.5;
        spawn_count = 8;
        sprintf(phase, "MORNING RUSH");
    } else if(hour >= 17 && hour < 19) {
        multiplier = 3.0;
        spawn_count = 10;
        sprintf(phase, "EVENING RUSH");
    } else if(hour >= 22 || hour < 6) {
        multiplier = 0.3;
        spawn_count = 1;
        sprintf(phase, "NIGHT TIME");
    } else {
        multiplier = 1.0;
        spawn_count = 3;
        sprintf(phase, "NORMAL HOURS");
    }

    printf("\n--- RESULTS ---\n");
    printf("Current Time: %02d:00\n", hour);
    printf("Time Phase: %s\n", phase);
    printf("Base Spawn Rate: 3 vehicles\n");
    printf("Multiplier: %.1fx\n", multiplier);
    printf("Actual Spawn Count: %d vehicles\n", spawn_count);

    if(hour >= 7 && hour < 9) {
        printf("Direction Bias: 70%% toward city center (Commuters arriving)\n");
    } else if(hour >= 17 && hour < 19) {
        printf("Direction Bias: 70%% away from city (Commuters leaving)\n");
    } else {
        printf("Direction Bias: Equal distribution in all directions\n");
    }
}

void emergency_vehicle_priority() {
    printf("\n=== FEATURE 4: EMERGENCY VEHICLE PRIORITY ===\n");

    int emergency_detected, distance_to_intersection;
```

```c
    printf("Is emergency vehicle detected? (1=Yes, 0=No): ");
    scanf("%d", &emergency_detected);

    if(emergency_detected) {
        printf("Enter distance to intersection (in meters): ");
        scanf("%d", &distance_to_intersection);

        printf("\n--- RESULTS ---\n");
        printf("Emergency Status: DETECTED 🚨\n");
        printf("Distance to Intersection: %d meters\n", distance_to_intersection);

        if(distance_to_intersection <= 500) {
            printf("\nAction: PRIORITY OVERRIDE ACTIVATED\n");
            printf("- All traffic lights: GREEN for emergency path\n");
            printf("- Other directions: RED (Hold traffic)\n");
            printf("- Expected clearance time: 15 seconds\n");
            printf("- Regular vehicles: Must yield\n");

            int speed = 80;
            float time_saved = (distance_to_intersection / 1000.0) / speed * 3600;
            printf("\nEstimated time to pass: %.1f seconds\n", time_saved);
            printf("Emergency Response Score: 95%% (Excellent)\n");
        } else {
            printf("\nAction: MONITORING (Not in priority range yet)\n");
            printf("- Priority activates at 500 meters\n");
            printf("- Currently: %d meters away\n", distance_to_intersection);
        }
    } else {
        printf("\n--- RESULTS ---\n");
        printf("Emergency Status: NONE\n");
        printf("Action: Normal traffic operation\n");
        printf("All intersections: Standard signal control\n");
    }
}

void run_full_simulation() {
    printf("\n=== FULL SYSTEM SIMULATION ===\n");

    int num_cycles;
    printf("Enter number of simulation cycles (1-10): ");
    scanf("%d", &num_cycles);

    for(int i = 0; i < 4; i++) {
        roads[i].vehicles = 10;
        roads[i].capacity = 50;
        roads[i].congestion = 0;
    }

    total_spawned = 0;
    total_completed = 0;
    int hour = 8;

    printf("\nRunning %d cycles starting from 08:00...\n", num_cycles);

    for(int cycle = 1; cycle <= num_cycles; cycle++) {
        printf("\n--- CYCLE %d | TIME: %02d:00 ---\n", cycle, hour);

        int spawn = 3;
        if(hour >= 7 && hour < 9) spawn = 8;
        else if(hour >= 17 && hour < 19) spawn = 10;
        else if(hour >= 22 || hour < 6) spawn = 1;

        total_spawned += spawn;
        printf("Spawned: %d vehicles\n", spawn);

        for(int i = 0; i < 4; i++) {
            roads[i].vehicles += spawn / 4;
            roads[i].congestion = ((float)roads[i].vehicles / roads[i].capacity) * 100;
        }

        float city_cong = 0;
        for(int i = 0; i < 4; i++) {
            city_cong += roads[i].congestion;
        }
        city_cong /= 4;
        printf("City Congestion: %.1f%%\n", city_cong);

        int cleared = 3;
        total_completed += cleared * 4;
        for(int i = 0; i < 4; i++) {
            roads[i].vehicles -= cleared;
            if(roads[i].vehicles < 0) roads[i].vehicles = 0;
        }

        hour++;
    }

    printf("\n=== SIMULATION COMPLETE ===\n");
    printf("Total Spawned: %d vehicles\n", total_spawned);
    printf("Total Completed: %d vehicles\n", total_completed);
    printf("Efficiency: %.1f%%\n", (float)total_completed / total_spawned * 100);
}

int main() {
    int choice;

    do {
        printf("  SMART TRANSPORT SIMULATION SYSTEM     \n");

        printf("  1. Adaptive Traffic Lights            \n");
        printf("  2. Congestion Calculation             \n");
        printf("  3. Rush Hour Simulation               \n");
        printf("  4. Emergency Vehicle Priority         \n");
        printf("  5. Run Full Simulation                \n");
        printf("  0. Exit                               \n");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                adaptive_traffic_lights();
                break;

            case 2:
                congestion_calculation();
                break;

            case 3:
                rush_hour_simulation();
                break;

            case 4:
                emergency_vehicle_priority();
                break;

            case 5:
                run_full_simulation();
                break;

            case 0:
                printf("  Thank you for using the system!       \n");
                printf("  Simulation terminated.                \n");
                break;

            default:
                printf("\n❌ Invalid choice! Please enter 0-5.\n");
        }

    } while(choice != 0);

    return 0;
}
```